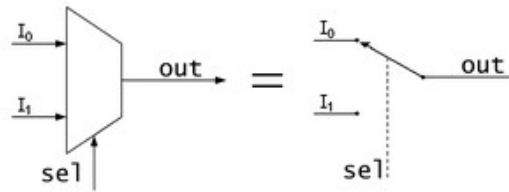


μProcessador 2 Multiplexação, Barramentos, Números: a ULA**Multiplexação**

Um mux é um seletor: ele escolhe uma das entradas possíveis para fornecer à saída.



(Figura: wikipedia)

Se quisermos um mux com 4 entradas ao invés de 2, a descrição fica:

```
entity mux4x1 is
  port( sel0, sel1      : in std_logic;
        entr0, entr1, entr2, entr3 : in std_logic;
        saida          : out std_logic
  );
end entity;
```

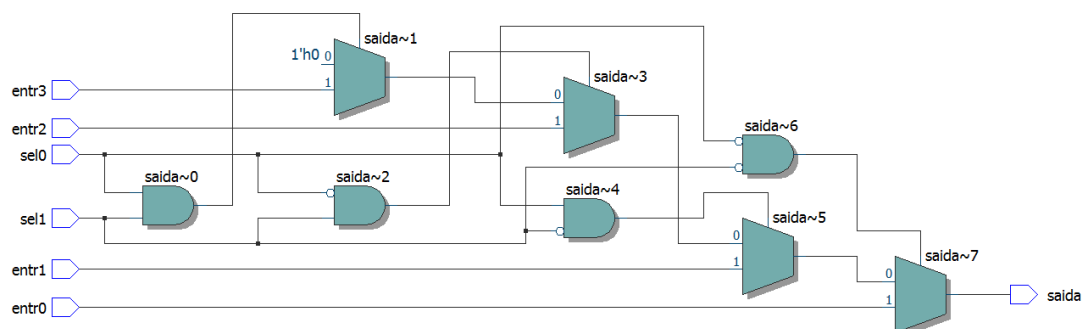
Observe a versão reduzida da tabela verdade do mux 4x1 acima.

sel1	sel0	saida
0	0	entr0
0	1	entr1
1	0	entr2
1	1	entr3

Em VHDL há diversas maneiras de fazer multiplexação. Vamos ficar com a seguinte:

```
architecture a_mux4x1 of mux4x1 is
begin
  saida <=  entr0  when sel1='0' and sel0='0' else
            entr1  when sel1='0' and sel0='1' else
            entr2  when sel1='1' and sel0='0' else
            entr3  when sel1='1' and sel0='1' else
            '0';    -- esse '0' é pra quando "der pau" em sel1 ou sel0
end architecture;
```

Lembre-se: isso não é programação, isso é um circuito! Abaixo vemos o RTL resultante deste trecho.

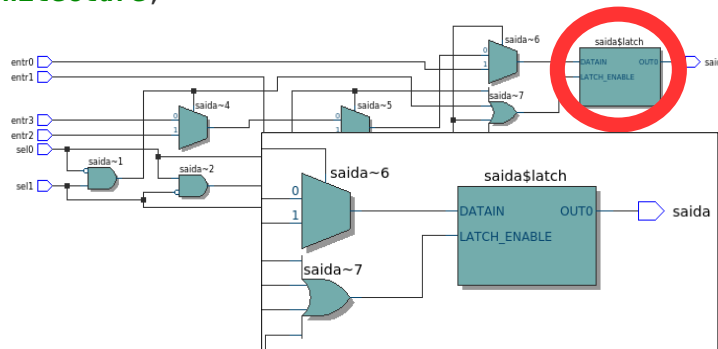


Mas pra que serve aquele “else '0';” da última linha? Não dá pra retirar?

Na verdade não... Lembre-se que o IEEE 1164 tem *nove* estados lógicos possíveis e *you* deve *cobrir todos os casos possíveis*. Se levássemos em conta apenas o 'U' (não inicializado) e o 'X' (desconhecido) além de 0 e 1, teremos *dezesseis* casos, se eu contei direito: 00, 01, 10, 11, 0U, U0, 1U, U1, 0X, X0, 1X, X1, XX, UX, XU e UU. O “else zero” cobre os outros doze deste exemplo, e mais ainda¹.

Por uma esquisitice do VHDL, quando os casos não são totalmente cobertos, ele coloca um *registrador* para os valores, ou seja, vai inserir um *latch* no meio do nosso circuito...(!) Veja abaixo o VHDL, caso você esqueça do “else '0';”, e o circuito resultante em RTL.

```
architecture a_mux4x1 of mux4x1 is
begin
    saida <=      entr0    when sel1='0' and sel0='0' else
                  entr1    when sel1='0' and sel0='1' else
                  entr2    when sel1='1' and sel0='0' else
                  entr3    when sel1='1' and sel0='1'; -- OPA! CADÊ O ELSE ZERO?
end architecture;
```



Ou seja:

ATENÇÃO: Numa estrutura when-else sempre termine com else '0';

Tá avisado.

►	Construa um multiplexador 8x1 com as restrições: as entradas 0, 1 e 5 estão sempre em '0', e as entradas 3 e 7 estão sempre em '1'. Portanto, apenas as entradas 2, 4 e 6 terão entradas variáveis, o que dá 3 pinos de dados e 3 pinos de seleção na entidade. Faça um <i>testbench</i> adequado.
---	---

Barramentos

Um barramento (*bus*) é apenas um feixe de sinais, um conjunto de fios. Aqui nesta disciplina vamos usá-los sempre como números inteiros, para facilitar.

Para isso, sempre inclua a biblioteca adicional padrão “numeric_std”² a partir de agora:

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all; -- para usarmos UNSIGNED
```

Ao definir um barramento, usamos a *keyword* “unsigned” (como em *unsigned integer*, inteiro não

¹ É possível simplesmente omitir a condição da última linha, como abaixo:

```
entr2    when sel1='1' and sel0='0' else
entr3;
```

A desvantagem é que se houver algum problema, veremos 'entr3' na saída; se ao invés disso observarmos a saída fixa em zero, é mais fácil localizar o *bug*.

² Há também a opção de usar a biblioteca depreciada *não padrão* “std_logic_arith”; mas **não use essa** pois *dá conflito* e é proprietária.

sinalizado, ou seja, não-negativo) e obrigatoriamente indicamos a largura do barramento em bits. Por exemplo, vamos fazer um multiplexador 4x1 de largura de 8 bits:

```
entity mux8bits is
    port(
        entr0 : in  unsigned(7 downto 0);
        entr1 : in  unsigned(7 downto 0);
        entr2 : in  unsigned(7 downto 0);
        entr3 : in  unsigned(7 downto 0);
        sel   : in  unsigned(1 downto 0); -- bits de seleção num só bus
        saida  : out unsigned(7 downto 0) -- lembre: sem ';' aqui
    );
end entity;
```

O tipo “unsigned(7 downto 0)” significa que o sinal tem 8 bits³ (8 pinos) e deve ser tratado como um inteiro não sinalizado, sem números negativos.⁴

A implementação é quase idêntica ao mux de 1 bit:

```
architecture a_mux8bits of mux8bits is
begin
    saida <=
        entr0 when sel="00" else -- observe as aspas duplas!
        entr1 when sel="01" else
        entr2 when sel="10" else
        entr3 when sel="11" else
        "00000000"; -- saida tem 8 bits, portanto 8 zeros
end architecture;
```

Números / Constantes

As constantes devem ser obrigatoriamente representadas por um vetor de bits⁵, representado por aspas duplas. Note que a extensão (*length*) deve bater! Veja exemplos para um sinal “saida” de 8 bits:

```
saida <= "00000010"; -- certo: cte. de 8 bits atribuída a sinal de 8 bits
saida <= "010";      -- pau: mismatch de largura
saida <= 65;          -- pau: mismatch de tipo unsigned <= inteiro;
```

O erro reportado na linha “saida <= 65;” é:

```
porta_e.vhdl:18:13: can't match integer literal with type
array type "unsigned"
ghdl: compilation error
```

pois o VHDL não sabe quantos bits a constante 65 possui⁶. Para resolver de forma simples, use o número equivalente em binário, “01000001”, com a largura esperada por “saida”.

Será fundamental recortarmos e combinarmos dados. Para isso, usamos o operador de concatenação “&” e o recurso de *slicing*, que obtém um trecho de bits.

Por exemplo, suponha dois barramentos de 8 bits:

```
signal x,y: unsigned(7 downto 0);
```

3 A keyword “downto” indica que a representação é *bit-level little endian*, ou seja, o bit 0 é o LSB (*least significant bit*, tem valor numérico 2⁰). No nosso exemplo, o bit 7 é o MSB (*most significant bit*) e tem valor 2⁷.

4 Note que para soma e subtração, podemos representar negativos em complemento de 2 usando este *unsigned* mesmo; o tal *signed* só é necessário para comparações e multiplicações sinalizadas.

5 Números inteiros, como 5 ou 122 por exemplo, são constantes sem especificação de largura em bits e portanto não podem ser usados diretamente com o tipo “unsigned.”

6 VHDL é uma linguagem *fortemente tipada*, o que na prática significa que ele requer conversões explícitas de tipos por parte do programador. Isso ajuda o código a chegar com menos erros na etapa de simulação.

Podemos combinar os bits dos sinais da forma que quisermos. Como ilustração simples, vamos isolar alguns trechos arbitrários:

- (a) O bit mais significativo (MSB) do sinal x é dado por $x(7)$
- (b) Os três bits menos significativos (LSB) de x são obtidos com $x(2 \text{ downto } 0)$
- (c) Para obter de b_5 a b_3 , basta fazer $x(5 \text{ downto } 3)$

Agora se quisermos por bobeira montar um número de 8 bits que começa com '0' e é seguido por (a), (b) e (c) nesta ordem, concatenamos tudo isso com o operador "&":

```
y <= '0' & x(7) & x(2 downto 0) & x(5 downto 3);
```

Portanto se x vale "01101010", y será "00010101". Se x for "11001100", teremos "01100001" em y .

Operações Aritméticas

Quando usamos o tipo "unsigned" e a biblioteca "numeric_std", dá pra fazer as paradinhas direto⁷. Por exemplo, vamos fazer um circuito com duas entradas que calcula tanto a soma quanto a subtração deles simultaneamente:

```
entity soma_e_subtrai is
  port (
    x,y      : in  unsigned(7 downto 0);
    soma,subt : out unsigned(7 downto 0)
  );
end entity;

architecture a_soma_e_subtrai of soma_e_subtrai is
begin
  soma <= x+y;
  subt <= x-y;
end architecture;
```

Muito mole.

Se quisermos saber se o x é maior do que o y e saber se o x é negativo ou não, podemos incluir na definição da entidade:

```
maior,x_negativo : out std_logic;
```

E na arquitetura basta colocarmos as linhas:

```
maior <= '1' when x>y else
        '0' when x<=y else
        '0';
x_negativo <= x(7);
```

Peralá, *negativo*? Sim, dá pra usar números sinalizados de 8 bits pra maioria das coisas (os circuitos são idênticos, pois realizam cálculos em módulo 2^8). Infelizmente, isso não funciona para a divisão. A comparação acima também só aceita números positivos (pois x e y são *unsigned*).

- | | |
|---|--|
| ► | Construa um <i>testbench</i> para o circuito "soma_e_subtrai" e o simule. Isso é importante para você observar corretamente a sintaxe de constantes e compreender a arquitetura acima.
Teste direito: faça casos relevantes tentando cobrir todas as operações. Por exemplo, se você testou $3+5$, não há necessidade de testar $13+15$, mas talvez seja interessante testar $100+100$ e $200+200$... Talvez não precise. Não esqueça de testar negativos, como $18+(-3)$. |
|---|--|

⁷ Isso não ocorre sem essa biblioteca, ok? Sem ela, precisa de uns malabarismos.

[Opcional/Recomendado] Múltiplos Arquivos Fonte

Para trabalhar com vários arquivos “.vhd”, siga a ideia do *testbench*: simplesmente declare o que você quer usar como um componente no outro arquivo. No terminal, rode *ghdl -a* para a análise em cada um dos fontes.⁸

Vamos rever o arquivo (besta) da porta E do lab #1, “porta.vhd”:

```
library ieee;
use ieee.std_logic_1164.all;

entity porta is
    port( in_a  : in std_logic;
          in_b  : in std_logic;
          a_e_b : out std_logic
        );
end entity;

architecture a_porta of porta is
begin
    a_e_b <= in_a and in_b;
end architecture;
```

Aí vamos fazer um outro, igualmente besta, que vai usar *três portas E*, uau! Vamos associar três portas de 2 entradas pra fazer uma porta E de 4 entradas. Batizemo-lo de “e_4_entradas.vhd”.

```
library ieee;
use ieee.std_logic_1164.all;

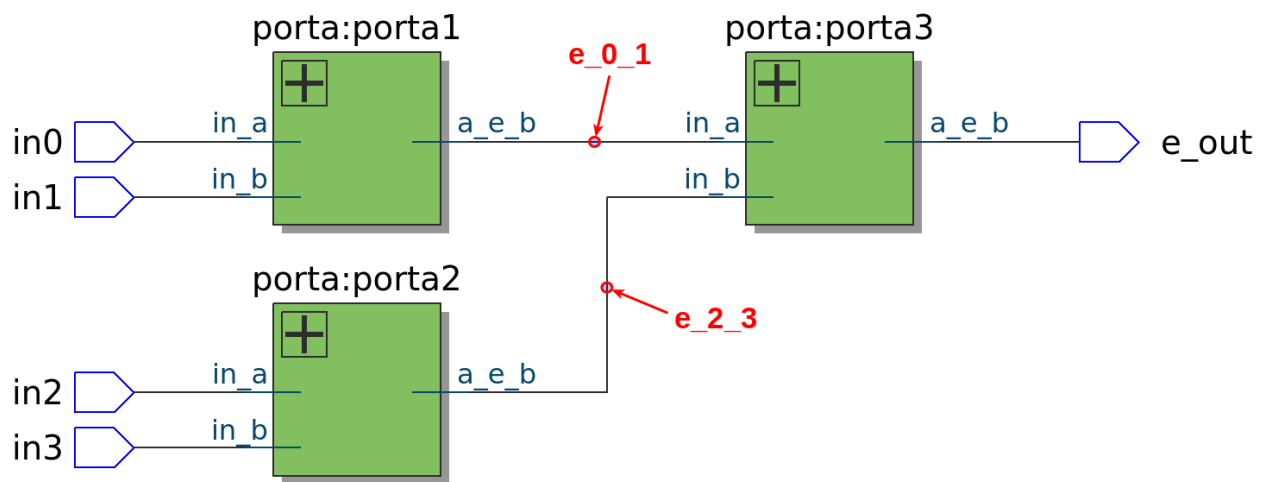
entity e_4_entradas is
    port( in0,in1,in2,in3: in std_logic;
          e_out: out std_logic
        );
end entity;

architecture a_e_4_entradas of e_4_entradas is
    component porta is
        port( in_a  : in std_logic;
              in_b  : in std_logic;
              a_e_b : out std_logic
            );
    end component;
    signal e_0_1, e_2_3: std_logic;
begin
    porta1: porta port map(in_a=>in0,in_b=>in1,a_e_b=>e_0_1);
    porta2: porta port map(in_a=>in2,in_b=>in3,a_e_b=>e_2_3);
    porta3: porta port map(in_a=>e_0_1, in_b=>e_2_3, a_e_b=>e_out);
end architecture;
```

Observe que apenas inserimos a interface do componente “porta” ali na arquitetura e o compilador se encarrega do resto. Na linha de comando, analise ambos os arquivos, dentro da mesma pasta, e rode a entidade do *testbench* final com, digamos, *ghdl -r e_4_entradas_tb --wave=result.ghw* ou algo assim.

Criei três portas E distintas: porta1, porta2 e porta3. Observe que com o “port map” nós conseguimos fazer a ligação entre os componentes sem gastar uma linha de código explícito. O circuito final é este:

8 Se você está usando o VHDLPlus, basta os arquivos estarem incluídos no projeto.



Os sinais indicados em vermelho, `e_0_1` e `e_2_3`, servem como “cola” para ligar as instâncias.

Por fim, lembrem-se de que, se encher o saco ficar selecionando sinais no *gtkwave*, podemos guardar a configuração de uma tela (sinais e zoom, essencialmente) usando o menu *File* => *Write Save File* para gravar a configuração de visualização (.gtkw).

Flags

A ULA produz normalmente quatro bits de sinalização para indicar o status da operação atual: são as chamadas *flags* N, C, Z e OV. Estas flags deverão, em laboratórios posteriores, serem guardadas em flip-flops. Elas irão servir para implementar saltos condicionais bem mais à frente. (O processador RISC-V que estudamos é uma das poucas exceções que não utilizam flags).

Cada equipe tem saltos condicionais sorteados para implementação. Consulte a tabela ao final deste documento para descobrir quais as flags você deverá implementar. As flags possíveis são:

- **Carry**, que indica se houve estouro em uma operação não sinalizada na ULA (ou seja, só com números positivos e zero);
- **Overflow**, que indica se houve estouro em uma operação sinalizada na ULA (ou seja, que inclui números negativos);
- **Zero**, que indica que o resultado da operação mais recente da ULA foi zero;
- Sinal (ou **Negativo**), que indica o sinal do resultado da operação mais recente da ULA, sendo apenas uma cópia do MSB.

Como funciona isso?

Suponha que temos um processador de 8 bits e vamos realizar uma instrução `ADD R1,R2,R3`. Os valores são $R2=11001000_2$ e $R3=01100100_2$; eles estão em binário porque o *processador em si não sabe se R2 e R3 são signed ou unsigned*, é o programador quem determina essa interpretação. A operação é $200_{10}+100_{10}$, ou em binário:

```

  11001000
+ 01100100
-----
(1) 00101100

```

O (1) entre parênteses indica o vai-um na saída do circuito somador (lembre dos somadores completos cascadeados); *este é o valor da carry flag*. Ele indica o estouro no caso em que consideramos R2 e R3 como unsigned. Dito de outra forma, o resultado exigiria um nono bit em 1; como os registradores têm 8 bits, ou seja, são feitos de 8 flip-flops, o resultado não cabe nele e

Cf=1. O número deveria ser $(1)00101100_2 = 300_{10}$ mas o resultado gerado é $00101100_2 = 44_{10}$.

Se, por outro lado, formos considerar R2 e R3 como signed, percebemos que R2 é negativo, R3 é positivo e o resultado é positivo (basta olhar o bit MSB: se b7 é 1, o número é negativo em complemento de 2). Portanto não há estouro e o valor da flag de **overflow** é 0. Só há estouro de números signed se somarmos dois números que têm um mesmo sinal e se o resultado tiver o sinal contrário.

Nos casos de subtração o raciocínio é análogo.

A **zero flag** estará valendo zero depois da execução da instrução, pois o resultado da operação é diferente de zero. Ela só vai ser ativada se a saída da ULA estiver em zero.

A flag de sinal ou a **flag de negativo** vale zero, pois o resultado é positivo quando interpretado em complemento de 2. É apenas a cópia do bit MSB, o b7.

Mais explicações sobre carry e overflow podem ser vistas no PDF de Complemento de 2 no Moodle.

Tarefa para Entregar: ULA

Nesta primeira entrega quero apenas o arquivo VHDL fonte e o VHDL do *testbench*.

Lembre-se que a entrega deste laboratório **vale nota** e não é pouca! Um atraso de poucos dias em um lab não vai impactar tanto a nota, mas *não coma bola!* No conjunto, vale bastante. Consulte as regras da disciplina no Moodle.

Não haverá apresentação pessoal em sala; apenas no laboratório #3 eu vou ver funcionando.

Especificação da ULA:

- duas entradas de dados de 16 bits;
- uma saída de resultado de 16 bits;
- duas ou mais saídas de sinalização de um bit (são as flags; implemente apenas as flags indicadas na tabela relativas ao seu sorteio);
- entradas para seleção da operação.

No mínimo 4 operações, incluindo:

- soma;
- subtração;
- duas outras quaisquer escolhidas pela equipe;
- será utilizada no projeto do processador (mas pode ser alterada à vontade posteriormente -- a validação poderá exigir operações especiais sorteadas);
- *não implemente divisão*, pois a implementação do compilador VHDL pode dar uns erros com divisão por zero (são contornáveis, mas melhor evitar).
- Um *testbench* que cobre todas as operações tem que ser entregue; inclua nele um conjunto de testes razoável, que cobre todos os casos de interesse, **incluindo números negativos nas entradas** (basta usar complemento de 2, com *unsigned* mesmo);
- Por favor dê nomes claros aos sinais (por ex., *op*, *selec*, *selec_op* ou *selecionaoperacao* e não "slo" ou "s")

Resumindo: entram dois dados de 16 bits, escolhe-se qual das 4 operações deve ser executada pela ULA e o resultado desta operação surge na saída de 16 bits. Entregue o *.vhd* e o *_tb.vhd*.

Ou seja, tem que fazer as operações e botar um mux na saída. É assim que se faz.

Avaliação

Você só pode usar o VHDL visto neste PDF e no anterior -- se quiser usar algo diferente, deve pedir explicitamente via email ao professor e explicar sua razão. Comandos não incluídos nos meus PDFs vão ser assumidos como sugestão de chatbots de IA e o laboratório será zerado sem possibilidade de recuperação.

Em especial, se aparecer *process* e/ou *if*, a nota será zero.

Apêndice: Tabela com Saltos Condicionais

Consulte suas instruções de saltos sorteadas para achar as flags que você precisa implementar. Por exemplo, se você pegou BNE e BHS, você terá que implementar as flags Z e C.

Sign	Suffix	Meaning	Flags
---	EQ	Equal	Z = 1
	NE	Not equal	Z = 0
	CS	Carry set (identical to HS)	C = 1
	CC	Carry clear (identical to LO)	C = 0
	MI	Minus or negative result	N = 1
	PL	Positive or zero result	N = 0
	VS	Overflow	V = 1
	VC	Not overflow	V = 0
	AL	Always. This is the default	-
Unsigned	HI	Higher	C = 1 and Z = 0
	HS	Higher or same	C = 1
	LS	Lower or same	C = 0 or Z = 1
	LO	Lower (identical to CC	C = 0
Signed	GT	Greater than	Z = 0 and N = V
	GE	Greater than or equal	N = V
	LE	Less than or equal	Z = 1 or N != V
	LT	Less than	N != V